36-40

問36: 範囲の二分探索

- 目的: ランク分け
- 変数
  - i: カウンタ
  - j: 二分探索の上限
  - k: 二分探索の下限
- ポイント
  - ランクは「x点以下 & y点より大きい」の形
    - 100以下80より大きい → 100以下 79以上
  - scoreRangeは点数の高い順に並んでいる→ 値が大きすぎれば、点数下限を上げる →インデックスは下がる

問37: ハッシュ法のトレース

- 目的: ハッシュ法を使って配列に格納する
- 変数
  - i: インデックス（ハッシュ関数で決まる）
- ポイント
  - hashArrayは全要素-1で初期化される
  - calcHash1で衝突が発生すれば、calcHash2でハッシュ値を求める

問38: ハッシュ(チェーン)法

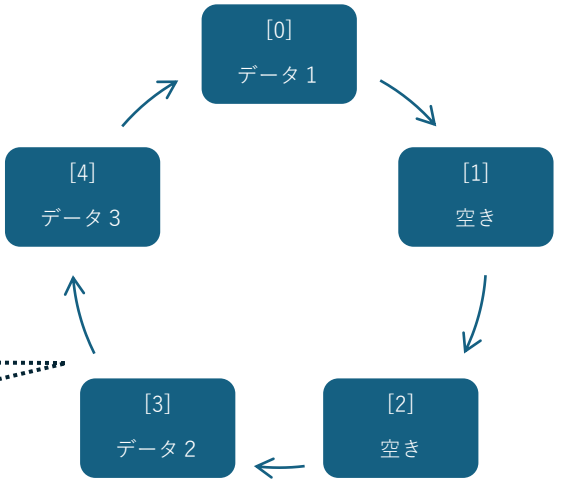
- 目的: ハッシュ探索
- 変数
  - i: ハッシュ配列のインデックス
  - tmp: コードを下の桁から一桁ずつ取り出す
  - cnt: ループカウンタ
- ポイント: 特殊な二次元配列

ハッシュ値が30になるコードを検索したときは、
bの操作 i ← hash[3,i]
によって、次のループで
i は-1になる

インデックス	・・・	28	29	30	・・・	55	56	57
コード		650	384	-1		465	183	708
データ		37	92	-1		22	54	85
シノニムのインデックス		-1	55	-1		57	-1	-1

問39: ハッシュ(オープンアドレス)法

- 目的: オープンアドレス法で、キーと値のペアを格納する
- 変数
  - hash: ハッシュ値, i: インデックス, pos: 格納先インデックス
- ポイント
  - 空欄[a] のあるwhileループは、格納先が決まらない間、回し続けるもの



ハッシュ値の箱が空きでなければ、順次、
となりの箱を見て、空いているものを探す

問40: 降順バブルソート

- 目的: 降順に並べ替え
- 変数
  - i: 確定領域, j: 未確定領域をスキャンする, flg: ループを抜けるか？
- ポイント
  - **降順**なので、
    - 配列の末尾から二つずつ比較する → j を 1 ずつ減らす
    - 配列の先頭から順序が確定してゆく→ i を 1 ずつ増やす